케라스 라이브러리



텐서플로우와 케라스

- 텐서플로우(TensorFlow)는 딥러닝 프레임워크의 일종이다. 텐서플로우는 내부적으로 C/C++로 구현되어 있고 파이썬을 비롯하여 여러 가지 언어에서 접근할 수 있도록 인터페이스를 제공한다.

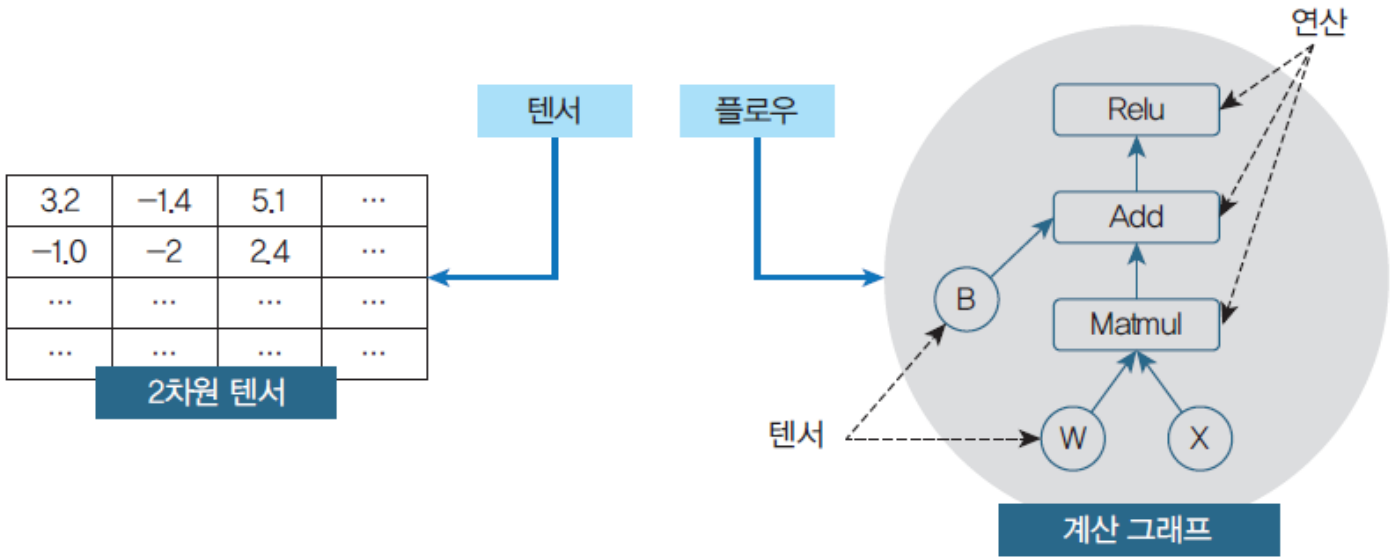
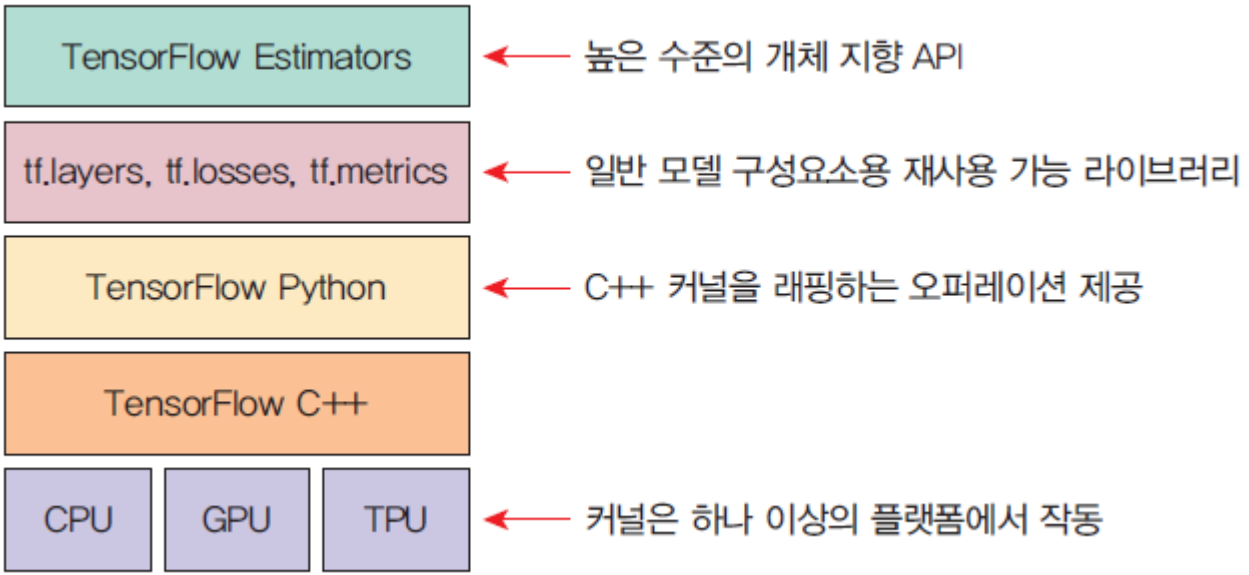


그림 7-4 텐서플로우의 개념

텐서플로우의 구조


TensorFlow Estimators
높은 수준의 개체 지향 API
tf.layers, tf.losses, tf.metrics
일반 모델 구성요소용 재사용 가능 라이브러리
TensorFlow Python
C++ 커널을 래핑하는 오퍼레이션 제공
TensorFlow C++
CPU
GPU
TPU
커널은 하나 이상의 플랫폼에서 작동



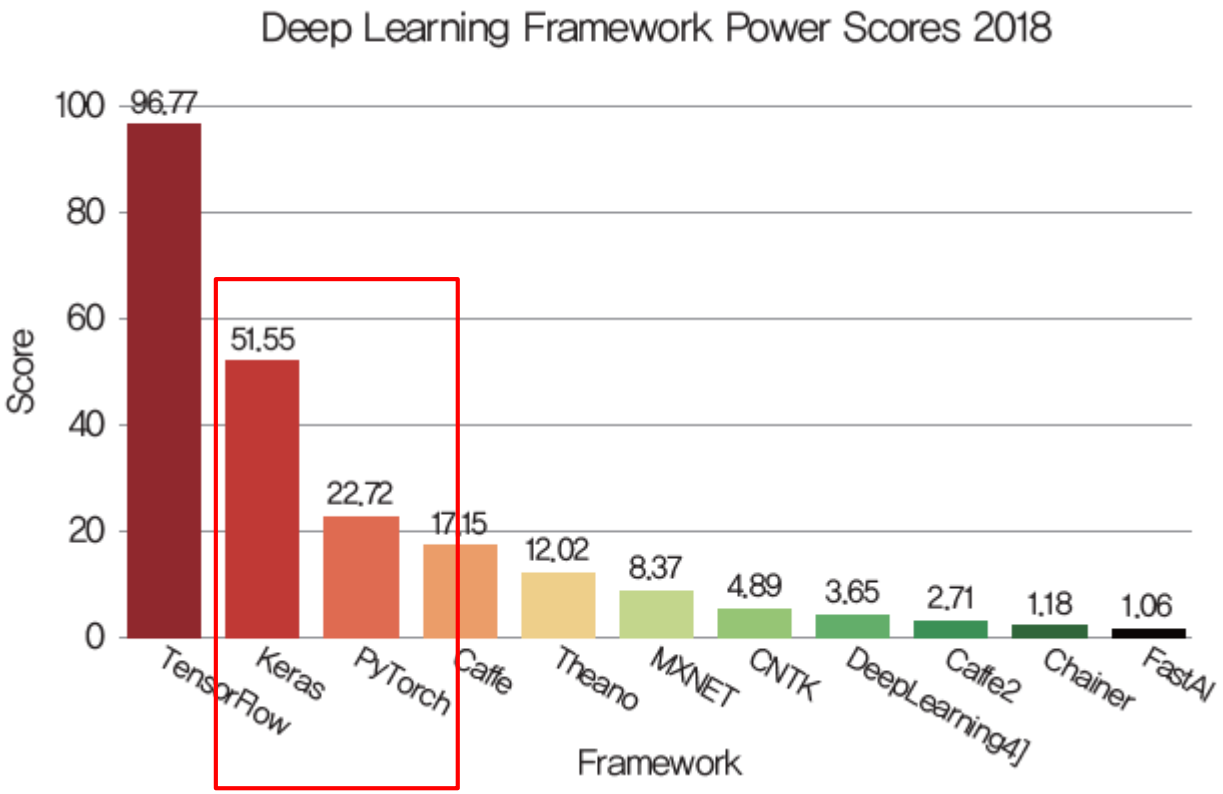
케라스

- 케라스는 파이썬으로 작성되었으며, 고수준 딥러닝 API이다. 케라스에서는 여러 가지 백엔드를 선택할 수 있지만, 아무래도 가장 많이 선택되는 백엔드는 텐서플로우이다
- 쉽고 빠른 프로토타이핑이 가능하다.
- 피드포워드 신경망, 컨볼루션 신경망과 순환 신경망은 물론, 여러 가지의 조합도 지원한다.
- CPU 및 GPU에서 원활하게 실행된다.





딥러닝 프레임워크


Deep Learning Framework Power Scores 2018
Score
100
80
60
40
20
0
96.77
51.55
22.72
17.15
12.02
8.37
4.89
3.65
2.71
1.18
1.06
TensorFlow
Keras
PyTorch
Caffe
Theano
MXNET
CNTK
DeepLearning4J
Caffe2
Chainer
FastAI
Framework



케라스로 신경망을 작성하는 절차

- 케라스의 핵심 데이터 구조는 모델(model)이며 이것은 레이어를 구성하는 방법을 나타낸다.
- 가장 간단한 모델 유형은 Sequential 선형 스택 모델이다. Sequential 모델은 레이어를 선형으로 쌓을 수 있는 신경망 모델이다

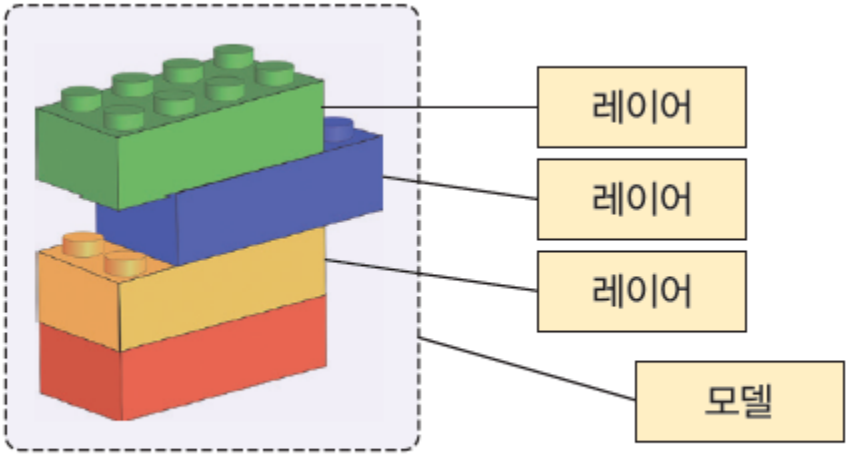
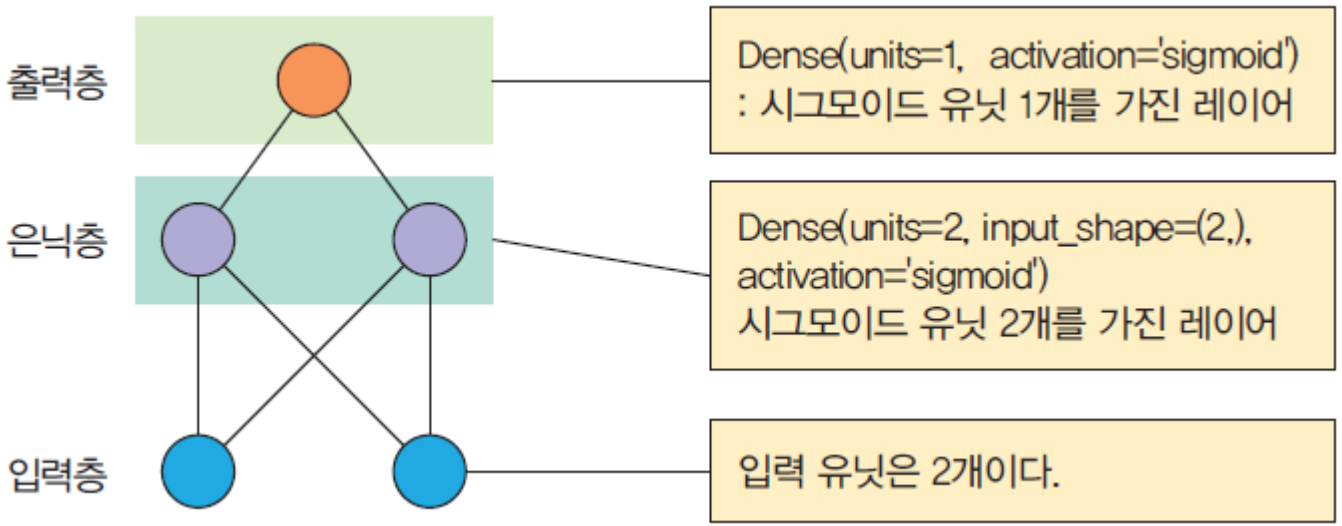


그림 7-5 케라스의 기본 개념



예제: XOR를 학습하는 MLP를 작성





훈련 데이터

	x1	x2		y
샘플 #1	0	0		0
샘플 #2	0	1		1
샘플 #3	1	0		1
샘플 #4	1	1		0



```
model = tf.keras.models.Sequential()


model.add(tf.keras.layers.Dense(units=2, input_shape=(2,), activation='sigmoid')) #①
model.add(tf.keras.layers.Dense(units=1, activation='sigmoid’)



model.compile(loss='mean_squared_error', optimizer=keras.optimizers.SGD(lr=0.3))


model.fit(X, y, batch_size=1, epochs=10000)


print( model.predict(X) )
```



model.summary()

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 2)                 6
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 3
=================================================================
Total params: 9
Trainable params: 9
Non-trainable params: 0
_________________________________________________________________
```



실행 결과

```
...
Epoch 10000/10000
4/4 [==============================] - 0s 748us/sample - loss: 9.7796e-04
[[0.02736092]
 [0.9704443 ]
 [0.9701309 ]
 [0.03734875]]
```

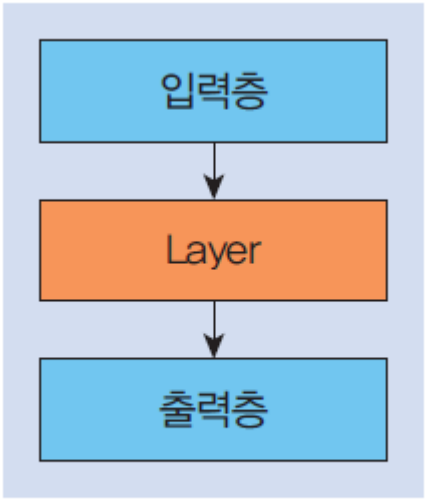


케라스를 사용하는 3가지 방법

(1) Sequential 모델을 만들고 모델에 필요한 레이어를 추가하는 방법이다.

```
model = Sequential()

model.add(Dense(units=2, input_shape=(2,), activation='sigmoid'))
model.add(Dense(units=1,  activation='sigmoid'))
```



Sequential

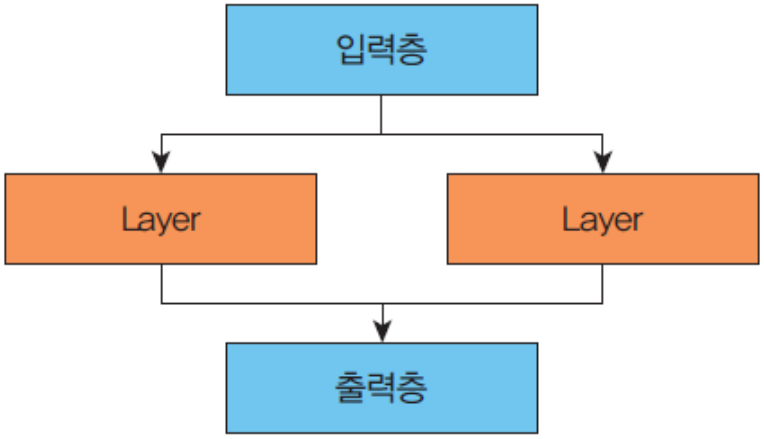


케라스를 사용하는 3가지 방법

(2) 함수형 API를 사용하는 방법

```
inputs = Input(shape=(2,))                          # 입력층
x = Dense(2, activation="sigmoid")(inputs)          # 은닉층 ①
prediction = Dense(1, activation="sigmoid")(x)      # 출력층

model = Model(inputs=inputs, outputs=prediction)
```





케라스를 사용하는 3가지 방법

(3) Model 클래스를 상속받아서 우리 나름대로의 클래스를 정의하는 방법

```
class SimpleMLP(Model):

    def __init__(self, num_classes):      # 생성자 작성
        super(SimpleMLP, self).__init__(name='mlp')
        self.num_classes = num_classes

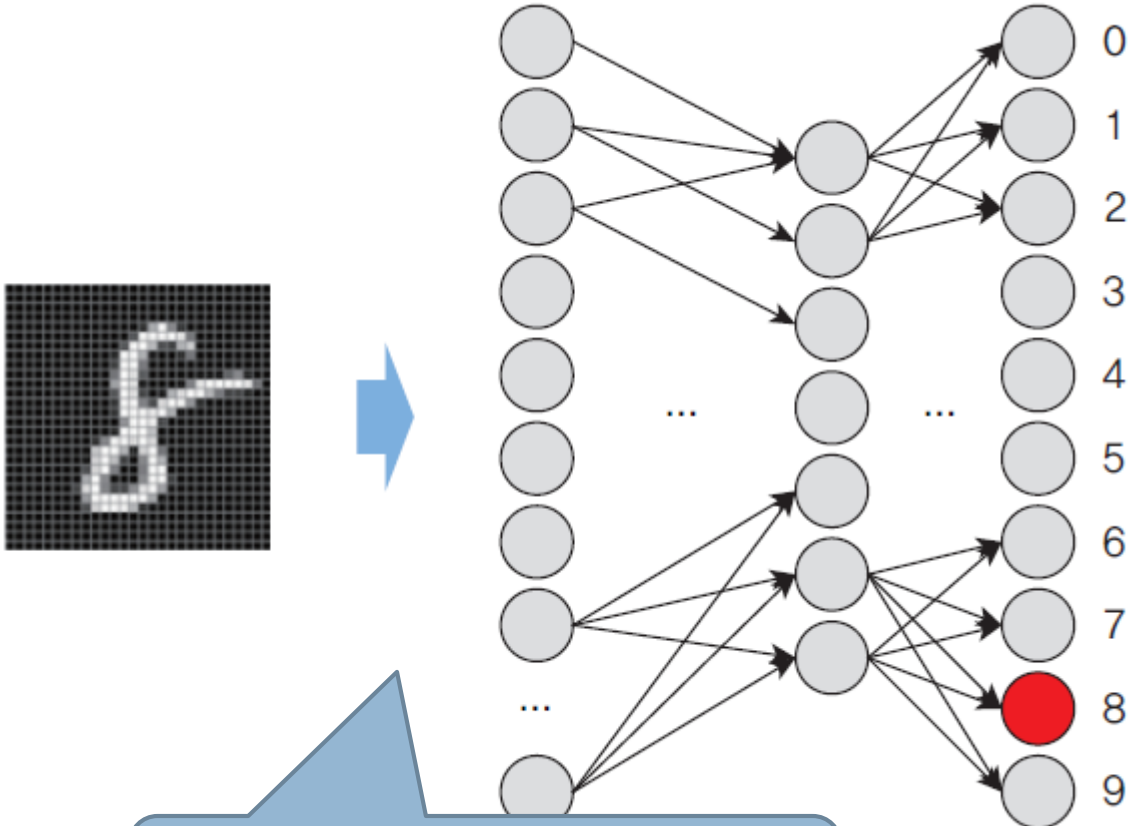
        self.dense1 = Dense(32, activation='sigmoid')
        self.dense2 = Dense(num_classes, activation='sigmoid')

    def call(self, inputs):               # 순방향 호출을 구현한다.
        x = self.dense1(inputs)
        return self.dense2(x)

model = SimpleMLP()
model.compile(...)
model.fit(...)
```



케라스를 이용한 MNIST 숫자 인식



MNIST 필기체 숫자 데이터 세트

- 1980년대에 미국의 국립표준 연구소(NIST)에서 수집한 데이터 세트으로 6만개의 훈련 이미지와 1만개의 테스트 이미지로 이루어져 있다



그림 7-7 MNIST 데이터



숫자 데이터 가져오기

```
import matplotlib.pyplot as plt
import tensorflow as tf

(train_images, train_labels), (test_images, test_labels)
                        = tf.keras.datasets.mnist.load_data()
```



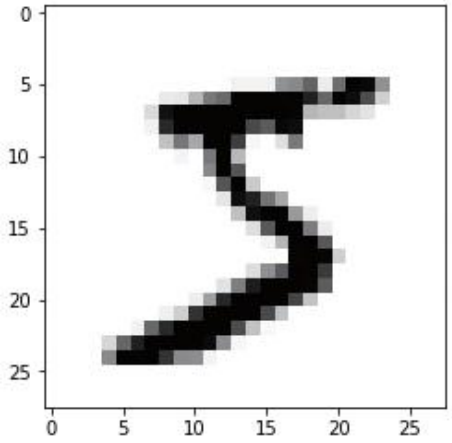
숫자 데이터 표시하기

```
>>> train_images.shape
(60000, 28, 28)

>>> train_label
array([5, 0, 4, ..., 5, 6, 8], dtype=uint8)

>>> test_images.shape
(10000, 28, 28)

>>> plt.imshow(train_images[0], cmap="Greys")
```

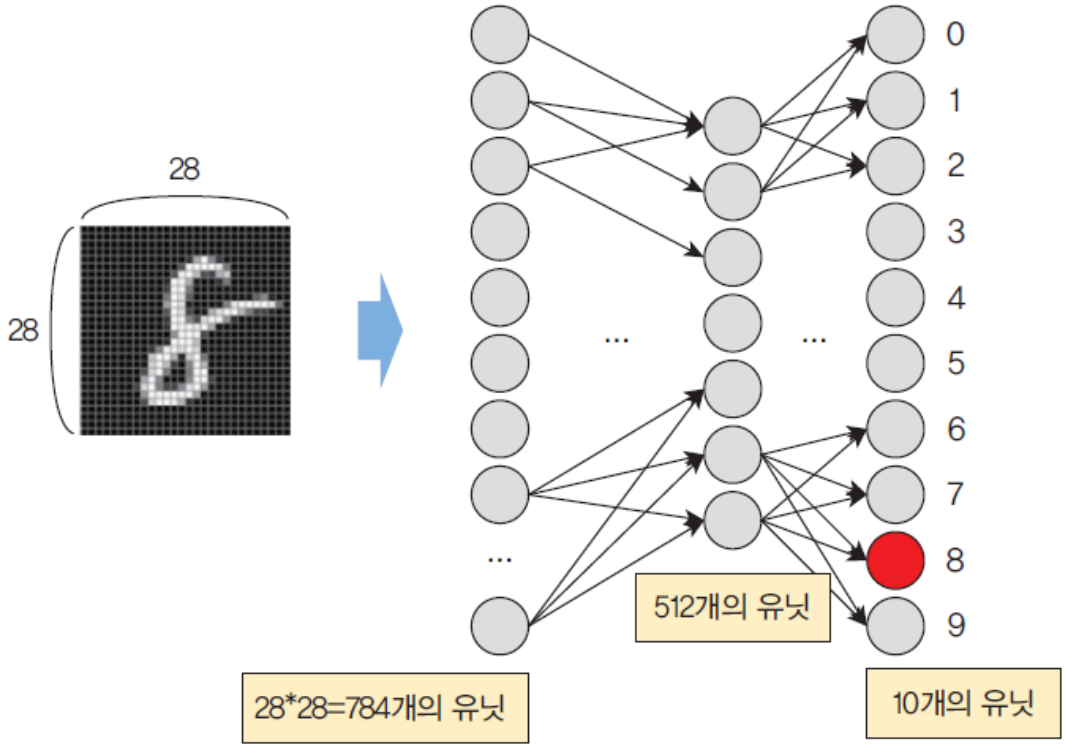




신경망 모델 구축하기

```
model = tf.keras.models.Sequential()

model.add(tf.keras.layers.Dense(512, activation='relu', input_shape=(784,)))
model.add(tf.keras.layers.Dense(10, activation='sigmoid'))
```





옵티마이저와 손실함수, 지표 등을 정의하는 컴파일 단계

```
model.compile(optimizer='rmsprop',
              loss='mse',
              metrics=['accuracy'])
```

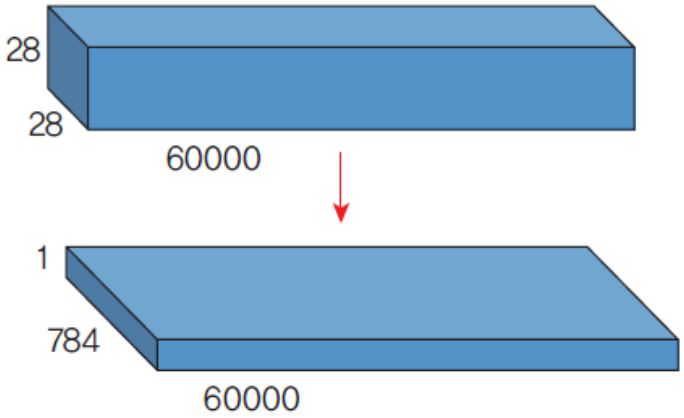
- 손실함수(loss function): 신경망의 출력과 정답 간의 오차를 계산하는 함수
- 옵티마이저(optimizer): 손실 함수를 기반으로 신경망의 파라미터를 최적화하는 알고리즘
- 지표(metric): 훈련과 테스트 과정에서 사용되는 척도



데이터 전처리

```
train_images = train_images.reshape((60000, 784))
train_images = train_images.astype('float32') / 255.0

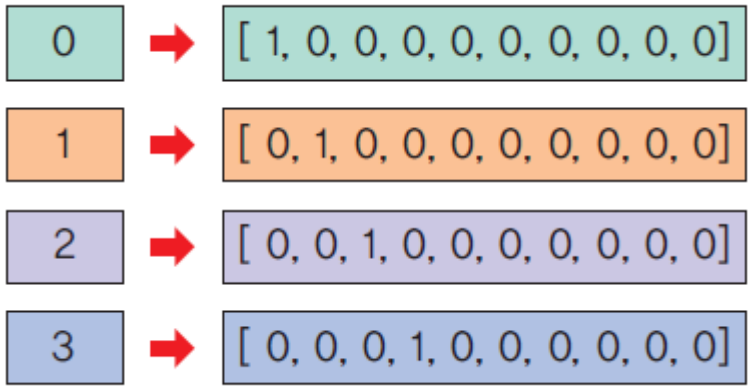
test_images = test_images.reshape((10000, 784))
test_images = test_images.astype('float32') / 255.0
```





정답 레이블 형태 변경(원핫 인코딩)

```
train_labels = tf.keras.utils.to_categorical(train_labels)
test_labels = tf.keras.utils.to_categorical(test_labels)
```





학습

```
model.fit(train_images, train_labels, epochs=5, batch_size=128)
```

```
Epoch 1/5
469/469 [==============================] - 2s 3ms/step - loss: 0.0158 - accuracy:
0.9168
...
Epoch 5/5
469/469 [==============================] - 2s 3ms/step - loss: 0.0027 - accuracy:
0.9867
```



테스트

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
print('테스트 정확도:', test_acc)
```

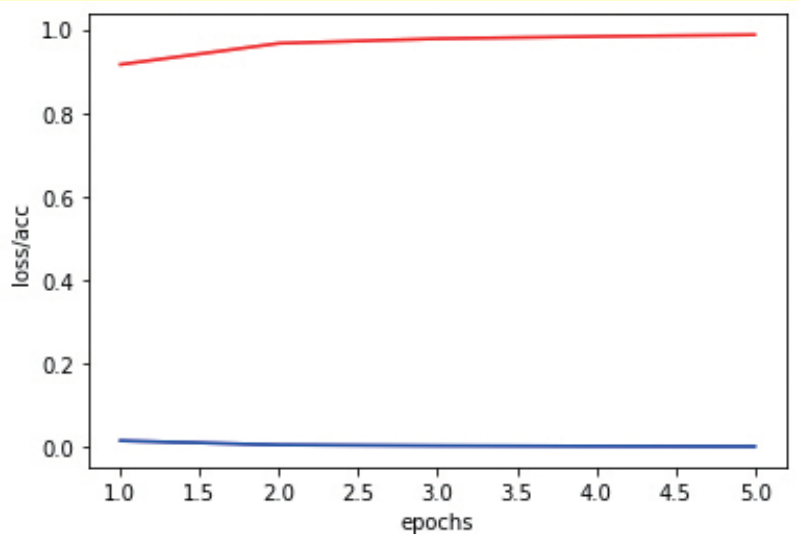
```
313/313 [==============================] - 0s 892us/step - loss: 0.0039 -
accuracy: 0.9788
테스트 정확도: 0.9787999987602234
```



그래프 그리기

```
history = model.fit(train_images, train_labels, epochs=5, batch_size=128)
loss = history.history['loss']
acc = history.history['accuracy']
epochs = range(1, len(loss)+1)

plt.plot(epochs, loss, 'b', label='Training Loss')
plt.plot(epochs, acc, 'r', label='Accuracy')
plt.xlabel('epochs')
plt.ylabel('loss/acc')
plt.show()
```



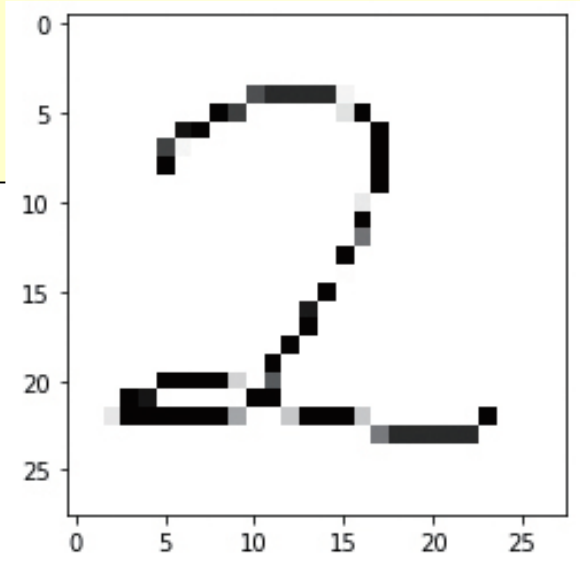


실제 이미지로 테스트하기

```
import cv2 as cv

image = cv.imread('test.png', cv.IMREAD_GRAYSCALE)
image = cv.resize(image, (28, 28))
image = image.astype('float32')
image = image.reshape(1, 784)
image = 255-image
image /= 255.0

plt.imshow(image.reshape(28, 28),cmap='Greys')
plt.show()
```





테스트

```
pred = model.predict(image.reshape(1, 784), batch_size=1)
print("추정된 숫자=", pred.argmax())
```

```
추정된 숫자= 2
```

도전문제

(1) 은닉층 유닛의 개수는 성능에 어떻게 영향을 끼치는가? 은닉층 유닛의 개수를 변경하면서 정확도가 어떻게 변하는지를 관찰해보자.

(2) 배치 크기를 변경하면서 학습의 정확도가 어떻게 변하는지를 관찰해보자.

(3) 은닉층의 활성화 함수를 relu에서 시그모이드 함수로 변경해보자. 학습의 정확도가 어떻게 변하는지를 관찰해보자.



케라스의 입력 데이터

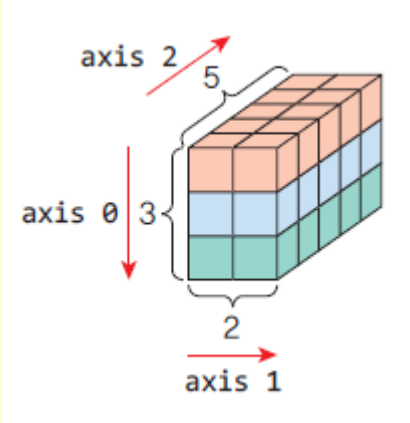
- 넘파이 배열:
- TensorFlow Dataset 객체: 크기가 커서, 메모리에 한 번에 적재될 수 없는 경우에 디스크 또는 분산 파일 시스템에서 스트리밍될 수 있다.
- 파이썬 제너레이터: 예를 들어서 keras.utils.Sequence 클래스는 하드 디스크에 위치한 파일을 읽어서 순차적으로 케라스 모델로 공급할 수 있다.



텐서

- 텐서는 다차원 넘파이 배열이다.
- 텐서는 데이터(실수)를 저장하는 컨테이너라고 생각하면 된다. 텐서에서는 배열의 차원을 축(axis)이라고 부른다.
- 예를 들어서 3차원 텐서는 다음과 같이 생성할 수 있다.

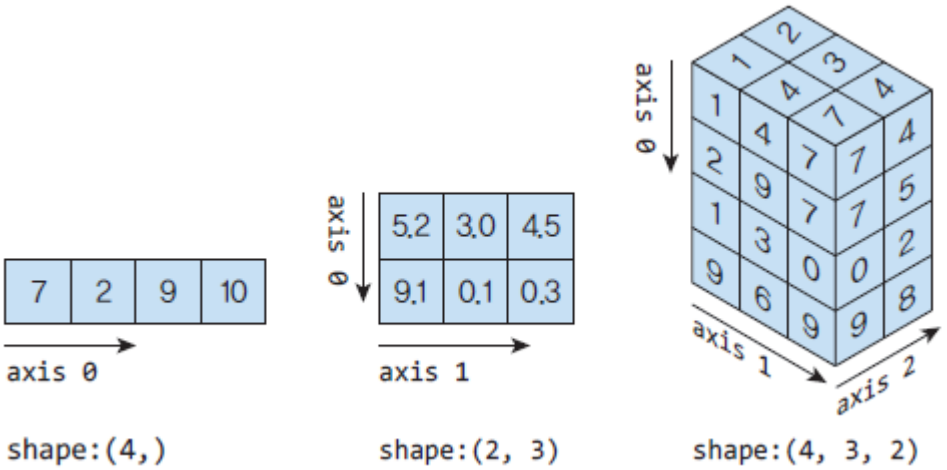
```
>>> import numpy as np
x = np.array(
  [[[0, 1, 2, 3, 4],
    [5, 6, 7, 8, 9]],
  [[10, 11, 12, 13, 14],
    [15, 16, 17, 18, 19]],
  [[20, 21, 22, 23, 24],
    [25, 26, 27, 28, 29]],])
>>> x.ndim
3
>>> x.shape
(3, 2, 5)
```





텐서의 속성

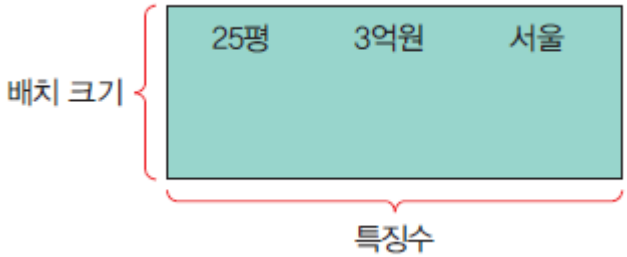
- 텐서의 차원(축의 개수): 텐서에 존재하는 축의 개수이다. 3차원 텐서에는 3개의 축이 있다. ndim 속성
- 형상(shape): 텐서의 각 축으로 얼마나 데이터가 있는지를 파이썬의 튜플로 나타낸 것이다.
- 데이터 타입(data type): 텐서 요소의 자료형.



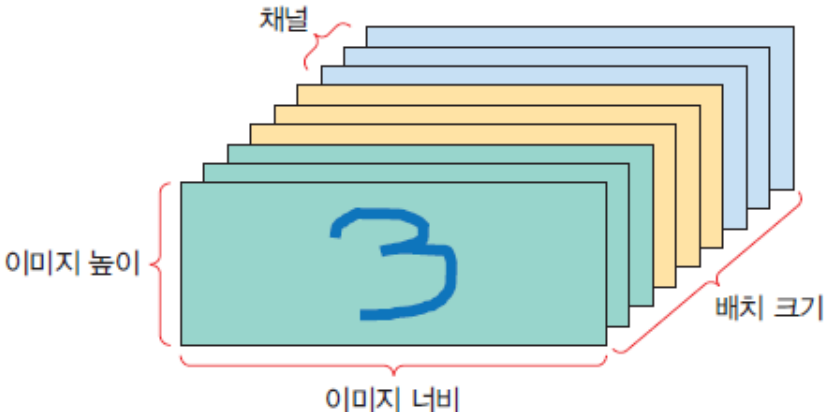


훈련 데이터의 형상

- 벡터 데이터: (배치 크기, 특징수)의 형상을 가진다.



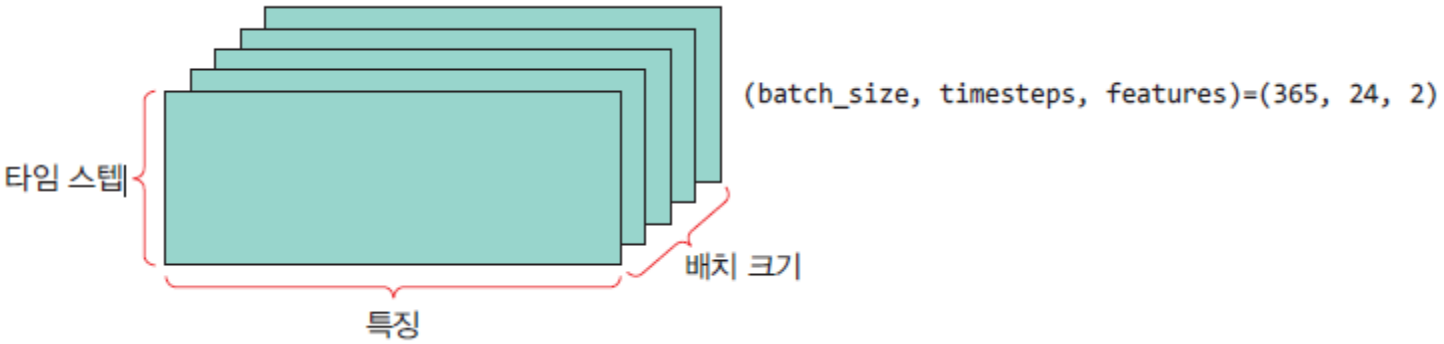
- 이미지 데이터: (배치 크기, 이미지 높이, 이미지 너비, 채널수) 형상의 4차원 넘파이 텐서에 저장된다





훈련 데이터의 형상

- 시계열 데이터: (배치 크기, 타입 스텝, 특징수) 형상의 3차원 넘파이 텐서에 저장된다.





케라스의 클래스들

- 모델: 하나의 신경망을 나타낸다.
- 레이어: 신경망에서 하나의 층이다.
- 입력 데이터: 텐서플로우 텐서 형식이다.
- 손실 함수: 신경망의 출력과 정답 레이블 간의 차이를 측정하는 함수이다.
- 옵티마이저: 학습을 수행하는 최적화 알고리즘이다. 학습률과 모멘텀을 동적으로 변경한다



Sequential 모델

- compile(optimizer, loss=None, metrics=None): 훈련을 위해서 모델을 구성하는 메소드
- fit(x=None, y=None, batch_size=None, epochs=1, verbose=1): 훈련 메소드
- evaluate(x=None, y=None): 테스트 모드에서 모델의 손실 함수 값과 측정 항목 값을 반환
- predict(x, batch_size=None): 입력 샘플에 대한 예측값을 생성
- add(layer): 레이어를 모델에 추가한다.



레이어 클래스들

- Input(shape, batch_size, name): 입력을 받아서 케라스 텐서를 생성하는 객체
- Dense(units, activation=None, use_bias=True, input_shape): 유닛들이 전부 연결된 레이어
- Embedding(input_dim, output_dim):



손실 함수

- MeanSquaredError: 정답 레이블과 예측값 사이의 평균 제곱 오차를 계산한다.
- BinaryCrossentropy: 정답 레이블과 예측 레이블 간의 교차 엔트로피 손실을 계산한다(예를 들어서 강아지, 강아지 아님).
- CategoricalCrossentropy: 정답 레이블과 예측 레이블 간의 교차 엔트로피 손실을 계산한다(예를 들어서 강아지, 고양이, 호랑이). 정답 레이블은 원핫 인코딩으로 제공되어야 한다.
- SparseCategoricalCrossentropy: 정답 레이블과 예측 레이블 간의 교차 엔트로피 손실을. 계산한다 (예를 들어서 강아지, 고양이, 호랑이). 정답 레이블은 정수로 제공되어야 한다.



측정 항목

- Accuracy: 정확도이다. 예측값이 정답 레이블과 같은 횟수를 계산한다.
- categorical_accuracy: 범주형 정확도이다. 신경망의 예측값이 원-핫 레이블과 일치하는 빈도를 계산한다.



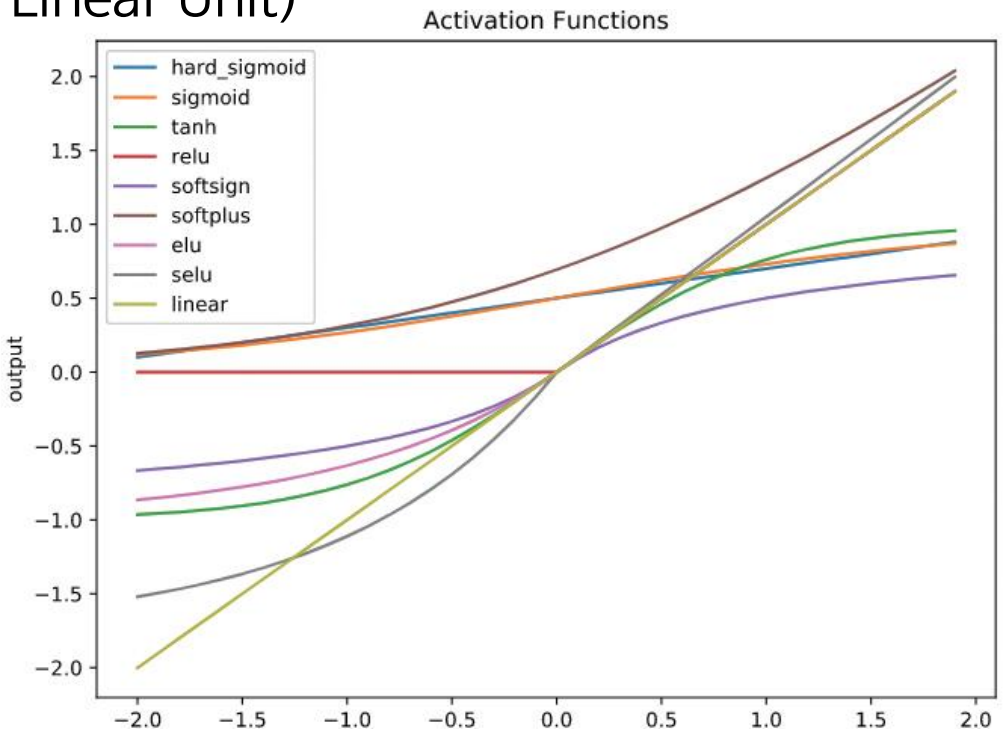
옵티마이저

- SGD: 확률적 경사 하강법(Stochastic Gradient Descent, SGD) . Nesterov 모멘텀을 지원한다.
- Adagrad: Adagrad는 가변 학습률을 사용하는 방법
- Adadelta: Adadelta는 모멘텀을 이용하여 감소하는 학습률 문제를 처리하는 Adagrad의 변형이다.
- RMSprop: RMSprop는 Adagrad에 대한 수정판이다.
- Adam: Adam은 기본적으로 (RMSprop + 모멘텀)이다.



활성화 함수

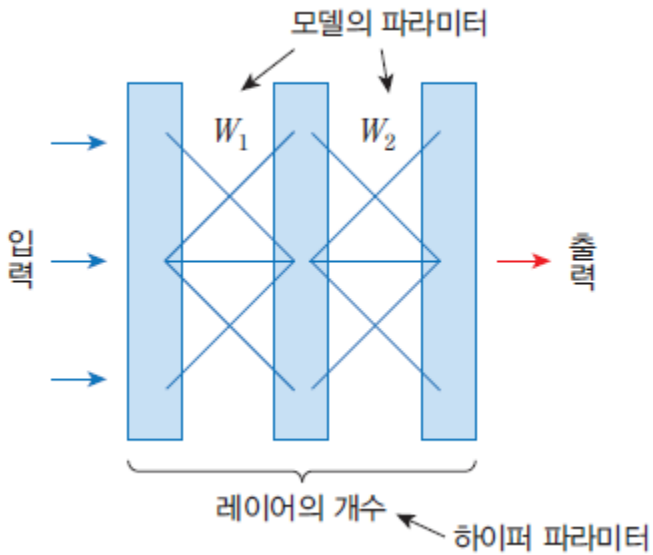
- sigmoid
- relu(Rectified Linear Unit)
- softmax
- tanh
- selu(Scaled Exponential Linear Unit)
- softplus





하이퍼 매개변수

- 학습률이나 은닉층을 몇 개로 할 것이며, 은닉층의 개수나 유닛의 개수는 누가 정하는 것일까? -〉 하이퍼 매개변수
- 즉 신경망의 학습률이나 모멘텀의 가중치, 은닉층의 개수, 유닛의 개수, 미니 배치의 크기 등이 하이퍼 매개변수이다





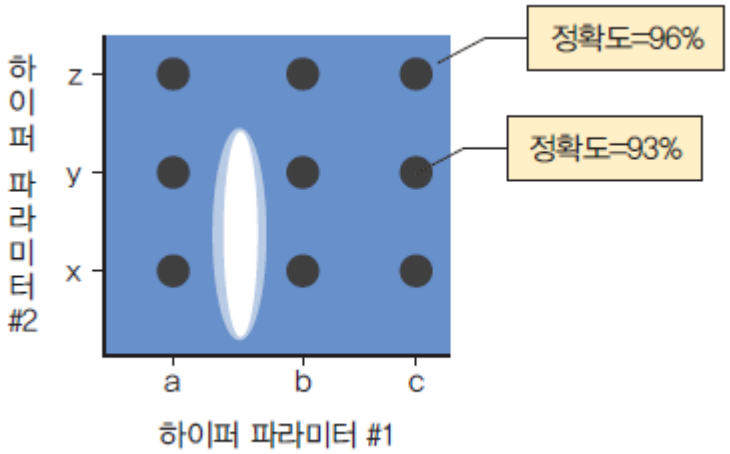
하이퍼 매개 변수를 찾는 방법

- 기본값 사용: 라이브러리 개발자가 설정한 기본값을 그대로 사용한다.
- 수동 검색: 사용자가 하이퍼 매개변수를 지정한다.
- 그리드 검색: 격자 형태로 하이퍼 매개변수를 변경하면서 성능을 측정하는 방법이다.
- 랜덤 검색: 랜덤으로 검색한다.



그리드 검색

- 각 하이퍼 매개변수에 대하여 몇 개의 값을 지정하면 이 중에서 가장 좋은 조합을 찾아주는알고리즘이다.
- sklearn 패키지에서 제공해주고 있기 때문에 손쉽게 사용할 수 있다





예제:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.model_selection import GridSearchCV
from tensorflow.keras.wrappers.scikit_learn import KerasClassifier

# 데이터 세트 준비
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.mnist.load_data()

train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype('float32') / 255

test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype('float32') / 255

train_labels = tf.keras.utils.to_categorical(train_labels)
test_labels = tf.keras.utils.to_categorical(test_labels)
```



예제:

케라스를 sklearn처럼 보이게 한다.

```
# 신경망 모델 구축
def build_model():
    network = tf.keras.models.Sequential()
    network = tf.keras.models.Sequential()
    network.add(tf.keras.layers.Dense(512, activation='relu', input_shape=(28 * 28,)))
    network.add(tf.keras.layers.Dense(10, activation='sigmoid'))

    network.compile(optimizer='rmsprop',
                    loss='categorical_crossentropy',
                    metrics=['accuracy'])
    return network

# 하이퍼 매개변수 딕셔너리
param_grid = {
              'epochs':[1, 2, 3],   # 에포크 수: 1, 2, 3
              'batch_size':[32, 64]             # 배치 크기: 32, 64
             }
```



예제:

```
# 케라스 모델을 sklearn에서 사용하도록 포장한다.
model = KerasClassifier(build_fn = build_model, verbose=1)

# 그리드 검색
gs = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=3,
    n_jobs=-1
)

# 그리드 검색 결과 출력
grid_result = gs.fit(train_images, train_labels)
print(grid_result.best_score_)
print(grid_result.best_params_)
```



실행결과

```
...
Epoch 3/3
938/938 [==============================] - 3s 4ms/step - loss: 0.0664 - accuracy:
0.9799
157/157 [==============================] - 0s 939us/step
0.968666672706604
{'batch_size': 64, 'epochs': 3}
```



Summary

- 풀배치는 훈련 데이터 세트를 모두 처리한 후에 가중치를 변경하는 것이다. 안정적이지만 확습속도는 늦다. SGD는 훈련 데이터 세트 중에서 랜덤하게 하나를 뽑아서 처리하고 가중치를 변경하는 방법이다. 속도는 빠르지만 불안정하게 수렴할 수 있다. 중간 방법이 미니 배치이다. 미니 배치에서는 일정한 수의 샘플을 뽑아서 처리한 후에 가중치를 변경한다.

- 학습률은 중요한 하이퍼 매개변수이다. 학습률은 적응적 학습 방법이 많이 사용된다. 즉 현재 그래디언트 값이나 가중치의 값을 고려하여 학습률이 동적으로 결정된다. 많이 사용되는 알고리즘으로 RMSprop나 Adam이 있다.

- 케라스를 사용하면 쉽게 신경망 모델을 구축할 수 있다. 가장 간단한 방법은 Sequential 모델을 생성하고 여기에 필요한 레이어들을 추가하는 방법이다. 많이 사용되는 레이어에는 Dense 레이어가 있다. Dense 레이어는 레이어 안의 유닛들이 다른 레이어의 뉴론들과 전부 연결된 형태의 레이어이다.

- 하이퍼 매개변수란 신경망 모델의 가중치나 바이어스와는 다르게, 개발자가 모델에 대하여 임의로 결정하는 값이다. 은닉층의 수나 유닛의 수, 학습률 등이 여기에 속한다. 그리드 검색을 사용하여서 최적의 값을 검색할 수도 있다



Q & A

